

```

        onPressed: () {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('修行榜单功能开发中')),
          );
        },
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
          backgroundColor: Colors.white.withOpacity(0.1),
          padding: const EdgeInsets.symmetric(vertical: 12),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        ),
        child: const Text('查看榜单'),
      ),
    ],
  ),
),
],
),
),
);
}

```

```

Widget _buildStatColumn(String value, String label) {
  return Column(
    children: [
      Text(
        value,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 4),
      Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12))
    ],
  );
}

```

```

/// 功能按钮行
Widget _buildActionButtons() {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 16),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        _buildActionButton(Icons.add_box_outlined, '补录功课', widget.onTapRecord),
        _buildActionButton(Icons.history, '查改记录', widget.onTapHistory),
        _buildActionButton(Icons.spa_outlined, '发愿回向', widget.onTapDedication),
        _buildActionButton(Icons.settings_outlined, '功课设置', widget.onTapSettings),
      ],
    ),
  );
}

```

```

Widget _buildActionButton(IconData icon, String label, VoidCallback? onTap) {
  return GestureDetector(
    onTap: onTap,
  );
}

```

```

        child: Column(
          children: [
            Container(
              width: 48,
              height: 48,
              decoration: BoxDecoration(
                color: const Color(0xFF2A2A2A),
                borderRadius: BorderRadius.circular(12),
              ),
              child: Icon(icon, color: Colors.white70, size: 22),
            ),
            const SizedBox(height: 8),
            Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12)),
          ],
        ),
      ),
    ),
  );
}

/// 发愿目标进度
Widget _buildGoalProgress(PracticeGoal goal) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text(
              '发愿目标: ${goal.currentCount} / ${_formatNumber(goal.targetCount)}',
              style: const TextStyle(color: Colors.white, fontSize: 14),
            ),
            Text(
              '${goal.progress}%',
              style: const TextStyle(color: Colors.white54, fontSize: 14),
            ),
          ],
        ),
        const SizedBox(height: 8),
        ClipRRect(
          borderRadius: BorderRadius.circular(4),
          child: LinearProgressIndicator(
            value: goal.progress / 100,
            minHeight: 6,
            backgroundColor: const Color(0xFF2A2A2A),
            valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFFE74C3C))
          ),
        ),
      ],
    ),
  );
}

/// 周/月统计图表
Widget _buildPeriodStats(List<DayStats> weeklyData) {
  return Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,

```

```

children: [
  Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(
        '本${_showWeekly ? "周" : "月"}: ${_showWeekly ? _service.weekTotal
: _service.monthTotal}, 本月: ${_service.monthTotal}',
        style: const TextStyle(color: Colors.white, fontSize: 14),
      ),
      Container(
        decoration: BoxDecoration(
          color: const Color(0xFF2A2A2A),
          borderRadius: BorderRadius.circular(16),
        ),
        child: Row(
          children: [
            _buildPeriodToggle('周', _showWeekly, () {
              setState(() => _showWeekly = true);
              _service.fetchWeeklyStats();
            }),
            _buildPeriodToggle('月', !_showWeekly, () {
              setState(() => _showWeekly = false);
              _service.fetchMonthlyStats();
            }),
          ],
        ),
      ),
    ],
  ),
  const SizedBox(height: 16),
  // 柱状图
  SizedBox(
    height: 120,
    child: weeklyData.isEmpty
      ? const Center(child: Text('暂无数据', style: TextStyle(color: Colors.white38)))
      : Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: weeklyData.map((day) => _buildBar(day)).toList(),
        ),
  ),
],
),
);
}

```

```

Widget _buildPeriodToggle(String label, bool isSelected, VoidCallback onTap) {
  return GestureDetector(
    onTap: onTap,
    child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      decoration: BoxDecoration(
        color: isSelected ? const Color(0xFFE74C3C) : Colors.transparent,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Text(
        label,
        style: TextStyle(
          color: isSelected ? Colors.white : Colors.white54,
          fontSize: 12,
        ),
      ),
    ),
  );
}

```

```

fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
    ),
  ),
);
}

Widget _buildBar(DayStats day) {
  // 计算柱子高度 (相对于最大值)
  final maxCount = _service.weeklyData.map((d) => d.count).fold(0, (a, b) => a
> b ? a : b);
  final ratio = maxCount > 0 ? day.count / maxCount : 0.0;
  final height = 80 * ratio + 10; // 最小高度10

  return Column(
    mainAxisAlignment: MainAxisAlignment.end,
    children: [
      Text(
        '${day.count}',
        style: TextStyle(
          color: day.count > 0 ? Colors.white : Colors.white38,
          fontSize: 11,
        ),
      ),
      const SizedBox(height: 4),
      Container(
        width: 28,
        height: height,
        decoration: BoxDecoration(
          gradient: day.count > 0
            ? const LinearGradient(
                colors: [Color(0xFFE74C3C), Color(0xFFC0392B)],
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
              )
            : null,
          color: day.count == 0 ? const Color(0xFF2A2A2A) : null,
          borderRadius: BorderRadius.circular(4),
        ),
      ),
      const SizedBox(height: 4),
      Text(
        day.date.substring(5), // MM-DD
        style: const TextStyle(color: Colors.white38, fontSize: 10),
      ),
    ],
  );
}

/// 连续打卡信息
Widget _buildStreakInfo(int consecutiveDays) {
  return Container(
    padding: const EdgeInsets.all(16),
    decoration: const BoxDecoration(
      border: Border(top: BorderSide(color: Color(0xFF2A2A2A))),
    ),
    child: Row(
      children: [
        const Icon(Icons.local_fire_department, color: Color(0xFFE74C3C), size
: 20),
        const SizedBox(width: 8),
        Text(

```

```

        '报数记录：您已连续打卡 $consecutiveDays 天',
        style: const TextStyle(color: Colors.white, fontSize: 14),
      ),
    ],
  ),
);
}

String _formatNumber(int number) {
  if (number >= 10000) {
    return '${(number / 10000).toStringAsFixed(0)}万';
  }
  return number.toString();
}
}

// =====
// 文件：lib/widgets/recaptcha_dialog.dart
// =====

import 'package:flutter/material.dart';
import 'package:flutter_inappwebview/flutter_inappwebview.dart';
import '../core/config/app_config.dart';

/// Firebase手机验证对话框
/// 使用托管的HTML页面 (Firebase JS SDK) 完成reCAPTCHA验证并发送验证码
class RecaptchaDialog extends StatefulWidget {
  final String phoneNumber;

  const RecaptchaDialog({
    super.key,
    required this.phoneNumber,
  });

  /// 显示对话框，发送验证码后返回sessionInfo
  /// 返回值：{'success': true, 'verificationId': '...'} 或 {'success': false, 'error': '...'}
  static Future<Map<String, dynamic>?> show(BuildContext context, {required String phoneNumber}) async {
    return showDialog<Map<String, dynamic>>(
      context: context,
      barrierDismissible: false,
      builder: (context) => RecaptchaDialog(phoneNumber: phoneNumber),
    );
  }

  @override
  State<RecaptchaDialog> createState() => _RecaptchaDialogState();
}

class _RecaptchaDialogState extends State<RecaptchaDialog> {
  bool _isLoading = true;
  String? _error;
  late InAppWebViewController _webViewController;

  // 使用托管的Firebase验证页面URL
  String get _verifyUrl {
    final encodedPhone = Uri.encodeComponent(widget.phoneNumber);
    // 使用Worker托管的验证页面
    return '${AppConfig.apiUrl}/phone-verify.html?phone=$encodedPhone';
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Dialog(
    backgroundColor: Colors.transparent,
    child: Container(
      width: 380,
      height: 500,
      decoration: BoxDecoration(
        color: const Color(0xFF1a1a2e),
        borderRadius: BorderRadius.circular(16),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.5),
            blurRadius: 20,
            offset: const Offset(0, 10),
          ),
        ],
      ),
    ),
    child: ClipRRect(
      borderRadius: BorderRadius.circular(16),
      child: Column(
        children: [
          // 标题栏
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
            decoration: const BoxDecoration(
              border: Border(
                bottom: BorderSide(color: Colors.white10),
              ),
            ),
            child: Row(
              children: [
                const Icon(Icons.security, color: Colors.white70, size: 20),
                const SizedBox(width: 8),
                const Expanded(
                  child: Text(
                    '安全验证',
                    style: TextStyle(color: Colors.white, fontSize: 16, font
Weight: FontWeight.w500),
                  ),
                ),
                IconButton(
                  icon: const Icon(Icons.close, color: Colors.white54, size:
20),
                  padding: EdgeInsets.zero,
                  constraints: const BoxConstraints(),
                  onPressed: () => Navigator.of(context).pop({'success': fal
se, 'error': '用户取消'})),
              ],
            ),
          ),
          // WebView
          Expanded(
            child: Stack(
              children: [
                InAppWebView(
                  initialUrlRequest: URLRequest(url: WebUri(_verifyUrl)),
                  initialSettings: InAppWebViewSettings(
                    javaScriptEnabled: true,

```

```
transparentBackground: true,
disableContextMenu: true,
supportZoom: false,
userAgent: 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15
_7) AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36',
),
onWebViewCreated: (controller) {
    _webViewController = controller;

    // 注册JavaScript处理器接收验证码发送结果
    controller.addJavaScriptHandler(
        handlerName: 'onCodeSent',
        callback: (args) {
            if (args.isNotEmpty && args[0] is Map) {
                final result = Map<String, dynamic>.from(args[0]);
                debugPrint('
```